

拥塞控制

在不知道路径容量时闭环试探

408 · NET-07

1 0. 本册定位

本 Topic 回答：大量独立发送者共享有限链路 with 队列、又看不到全局负载时，端点怎样用反馈调节 offered load，避免网络进入高排队、高丢包、低有效吞吐的失控状态？

本册拥有 congestion signal、cwnd、sssthresh、slow start、congestion avoidance、AIMD、fast retransmit/recovery 的 408 经典模型，以及效率、公平、时延和稳定性的权衡。

本册使用 TCP sequence/ACK/connection state，见传输层；可靠性与接收端流控分别不等同于 congestion control。

2 1. 根本问题：总输入可以超过共享服务能力

设某瓶颈链路服务率为 C ，多个流的总到达速率为 λ ：

- $\lambda < C$ ：队列通常能被清空；
- $\lambda \approx C$ ：小波动就会积累排队；
- $\lambda > C$ 持续存在：队列增长，最终丢包或时延失控。

朴素方案是每个 sender 都尽可能快地发送。单个 sender 看起来在提高自身吞吐，但所有 sender 的独立最优叠加会破坏共享资源：

More offered load → Queue growth → Delay/loss → Retransmission → Even more load

拥塞控制必须打断这个正反馈。

3 2. 拥塞是路径状态，不是单个 packet 的属性

发送端通常不知道：瓶颈在哪里、可用容量多少、同时有多少竞争流。它只能观察反馈：

- loss/timeout；
- duplicate ACK pattern；
- increasing RTT/queue delay；
- ECN 等显式标记；
- ACK 到达节奏。

然后更新本地发送约束：

Probe → Observe feedback → Infer path state → Adjust cwnd → Probe again

这是反馈控制，不是一次算出带宽后永久使用。

4 3. cwnd 的对象与单位

cwnd 是 sender 为该 connection 维护的 congestion window，限制网络中允许存在的未确认数据量。TCP 实际发送还受 receiver window：

$$FlightSize \leq \min(cwnd, rwnd).$$

408 题目有时用 MSS 个数表示窗口，有时用 byte。计算前必须写单位；不能把 $cwnd = 8$ 自动解释成 8 byte 或 8 segment。

若 sender 希望充分利用路径，理想在途量与路径 BDP 同量级；大幅超过时，多余数据主要进入 queue，而不是让传播管道变长。

5 4. Slow Start：在未知容量下快速探测

连接开始或严重拥塞后，若从 1 MSS 线性增加，找到高速路径容量太慢。Slow Start 借 ACK clock 让 cwnd 在每个 RTT 约翻倍：

```
cwnd = 1 MSS
one RTT of ACKs -> about 2 MSS
next RTT         -> about 4 MSS
next RTT         -> about 8 MSS
...
```

名称中的“slow”是相对一开始直接发送巨大 burst；它的增长在 RTT 维度上是指数级。

当 cwnd 到达 ssthresh 附近，切换到 congestion avoidance。ssthresh 不是路径真实容量，而是根据历史拥塞更新的本地分界估计。

6 5. Congestion Avoidance：加性试探

接近已知可用范围后，继续指数增长会频繁过冲。经典 congestion avoidance 让 cwnd 每 RTT 大约增加 1 MSS：

$$\Delta cwnd \approx 1 \text{ MSS} / \text{RTT}.$$

每个 ACK 的具体增量可按当前 cwnd 分摊，使一整个窗口被确认后累计约 1 MSS。核心不是背某个实现公式，而是识别增长从 multiplicative 变为 additive。

7 6. Congestion event 后为什么要乘性下降

若检测到拥塞仍只减一个固定小量，多个 flow 可能长时间把总负载维持在瓶颈之上。经典 AIMD：

Additive Increase + Multiplicative Decrease

用缓慢线性增长持续探测剩余容量，用比例下降快速释放大量在途负载。AIMD 在理想同质条件下还具有向效率与公平状态收敛的几何直觉，但真实 RTT、MSS、应用模式和算法差异会影响公平性。

8 7. Timeout 与 duplicate ACK 表示不同强度的证据

8.1 7.1 Timeout: 反馈闭环严重中断

RTO 到期意味着对应数据在足够长时间内没有获得预期确认。经典 408/Reno 模型把它视为较严重拥塞：更新 ssthresh，将 cwnd 降到较小初值，重新 slow start。

8.2 7.2 Three duplicate ACKs: 后续数据仍在通过

连续 duplicate ACK 表明接收端反复看见同一个缺口，但缺口之后的一些 segments 已到达。这通常比 timeout 提供更细的信息：路径仍在交付数据，只是某个 segment 可能丢失。

经典模型由此产生：

- fast retransmit: 不等 RTO，重传推定丢失 segment；
- fast recovery: 不把发送速率退回最初状态，而按相对温和方式降低并继续。

具体 cwnd/ssthresh 数值更新必须服从题目指定的 Tahoe/Reno 教材模型；现代 TCP 实现有多种 congestion-control algorithms，不能把一张 408 状态图冒充所有工程实现。

9 8. ACK clock: 反馈不仅说“收到”，还提供节奏

当 ACK 随数据穿过瓶颈返回，ACK 到达节奏近似反映路径的交付节奏。sender 按 ACK 释放新数据，可以避免无节制 burst。

但 ACK compression、delayed ACK、reverse-path congestion 等会扭曲时钟。ACK clock 是有用反馈机制，不是精确的全局容量测量仪。

10 9. Queue 的双重身份

Queue 不是纯粹坏事：短队列吸收 burst，使链路持续忙碌。问题是持久过量队列：

- latency 增大；
- timeout 和交互性能恶化；
- buffer overflow 导致 loss；
- 新流和短流被已有大队列拖累。

因此优化目标不是“绝不排队”或“永不丢包”，而是在 utilization、delay、loss、fairness 和 stability 之间取舍。

11 10. Network-assisted 与 end-to-end feedback

拥塞控制先分“事前约束”与“事后反馈”。事前/open-loop 策略在观察本次拥塞前限制系统行为，例如接纳控制、资源预留、流量整形、调度与丢弃策略；反馈/closed-loop 策略检测 queue、delay、loss 或显式标记，再把信号送回能降低 offered load 的主体。前者降低进入危险区的概率，后者适应实际运行状态；二者可以同时存在。

再按反馈 Owner 分层：router/network layer 可调度队列、丢弃或标记 packet，也可在特定体系中反馈显式速率；transport endpoint 根据 ACK/loss/ECN/RTT 等调整发送窗口或速率。网络层“发现/标

记拥塞”不自动完成端点减速，端点算法也不能直接观察每台 router 的完整队列真相。

- loss-based control: 从丢包/ACK 推断拥塞，部署简单但信号较晚；
- ECN: 网络设备在丢包前标记拥塞，端点仍负责降低发送；
- delay-based control: 从 RTT/queue delay 变化提前推断，但需要区分路径基线和噪声；
- explicit rate 等更强网络协助模型直接给出速率或资源信息，但需要基础设施支持。

408 核心以经典 TCP loss-based 模型为主；扩展用于解释反馈设计空间，不替代考试指定算法。

因此网络层拥塞与 TCP 拥塞控制不是两种互斥现象：前者描述共享 router/link 的资源状态与网络可提供的控制动作，后者是端点借助有限反馈调节 offered load 的一种闭环实现。

12 11. 408 状态推进：把题设算法写成转移函数

先固定单位 (byte 或 MSS)、初值、‘ssthresh’、ACK 粒度与题设版本。然后对每个事件应用：

事件	进入前状态	经典教材动作	停止/下一分支
new ACK, $cwnd < ssthresh$	slow start	每 ACK 增长；一 RTT 近似翻倍	达阈值转 avoidance
new ACK, $cwnd \geq ssthresh$	avoidance	一 RTT 近似增加 1 MSS	继续探测
timeout	任一发送阶段	$ssthresh \leftarrow$ 题设比例； $cwnd \leftarrow$ 小初值	重传并回 slow start
three duplicate ACKs, Tahoe	检测单缺口	fast retransmit；窗口降到小初值	slow start
three duplicate ACKs, Reno	检测单缺口	fast retransmit；进入/维持 fast recovery	new ACK 后回 avoidance

具体加一、减半、初始窗口和 recovery 膨胀数值以题目定义为准。手册拥有的是“状态、事件、反馈、转移”模型，不把某一版本常数写成所有 TCP 的永恒事实。

轮次与逐 ACK 不能混算

若题目说 slow start 每收到一个确认新数据的 ACK 增加 1 MSS，则一个窗口内的多个 ACK 累计造成下一 RTT 可发送量近似翻倍；不能把“每 ACK 加 1”误写成“每 ACK 翻倍”。题目若直接按 RTT 给窗口序列，则不要再重复逐 ACK 累加。

13 12. 概念边界

概念 A	≠ 概念 B	真正区别与题目信号	混淆后果
Congestion	≠ Receiver overflow	路径共享资源过载；接收应用/缓冲不足	cwnd 与 rwnd
cwnd	≠ rwnd	sender 的路径推断；receiver 的容量通告	错判谁持有和谁
ssthresh	≠ Bottleneck capacity	本地算法阶段阈值；路径容量是外部动态状态	把阈值当精确带
Timeout	≠ Certain congestion	timeout 是强推断，也可能受非拥塞丢失/延迟影响	把观测当全局事

概念 A	≠ 概念 B	真正区别与题目信号	混淆后果
Duplicate ACK	≠ New data ACK	前者重复声明同一连续前缀缺口	ACK 数与窗口
Flow rate	≠ Goodput	前者可含重传/首部；后者只数应用有效数据	拥塞时“发送更
Slow start	≠ Linear slow growth	RTT 维度近似指数增长	窗口轮次推演错
Classic Reno model	≠ All modern TCP	408 模型是一个算法实例	用旧状态图断言

14 12. 做题调用协议

1. 标出 cwnd、sssthresh、rwnd 的单位和初值；
2. 画 RTT 轮次或逐 ACK 事件，按题目粒度更新；
3. 判断当前处于 slow start、avoidance 还是 recovery；
4. 区分 timeout 与 duplicate ACK 触发的教材分支；
5. 计算实际发送上限时取 $\min(\text{cwnd}, \text{rwnd})$ 并扣除 in-flight；
6. 明确题目采用 Tahoe、Reno 或自定义规则；
7. 用 BDP、queue 和 goodput 检查窗口变化是否有物理意义。

15 13. 贯穿母例：为什么 1、2、4、8 后不能永远翻倍

假设瓶颈在途与队列可暂时容纳约 10 MSS。sender 从 1 MSS slow start：

```
1 -> 2 -> 4 -> 8: path may absorb
8 -> 16: offered flight overshoots available capacity
-> queue grows / loss signal appears
-> sender reduces cwnd and records sssthresh
-> later probes additively near the observed boundary
```

Slow start 解决“完全不知道容量时怎样快速接近”；AIMD 解决“接近边界后怎样持续试探并在过冲时退让”。二者不是两套互不相关的背诵规则，而是探索阶段不同。

16 14. 高频 First Divergence

- 把 cwnd 当 receiver 通告值：状态 owner 错；
- 每收到一个 ACK 就让 cwnd 翻倍：混淆逐 ACK 增量与逐 RTT 效果；
- 到 sssthresh 后停止增长：忘记 avoidance 仍在探测；
- timeout 与三次重复 ACK 使用同一分支：丢失证据强度分层；
- 只算发送速率不算 goodput：重传负载被当成有效数据；
- 未读题设就套 Reno：没有确认教材模型。

17 15. 一页压缩与复原问题

Probe capacity → Read ACK/loss/ECN/delay → Infer congestion → Adjust cwnd → Probe again

1. 为什么 congestion control 必须是闭环而不是固定速率？
2. Slow Start 为什么既“慢”又近似指数增长？
3. AIMD 的加性与乘性分别解决什么？
4. duplicate ACK 为什么通常比 timeout 提供更细的路径信息？
5. Queue 在什么范围内有益，何时开始伤害系统？

18 16. 来源与校正说明

- 归档材料没有独立拥塞控制正文，本册不是从术语表复制而来；
- cwnd、rwnd 共同限制在途数据，以及 slow start/congestion avoidance/fast retransmit/recovery 的经典边界依据 RFC 5681 校正；
- 具体窗口初值和现代算法会演进，本册将工程现实与 408 指定模型分层，不把版本相关常数写成永恒机制。